

Numerical Gradient Checking, Worked Out by Hand

Finite differences as a unit test for your backward pass

A complete numerical walk-through of how you confirm a hand-derived gradient is correct *without trusting any calculus*. We probe the function with two nearby evaluations, form a finite-difference estimate of the derivative, and compare it against the analytic gradient. We watch the error shrink like ε^2 as the step shrinks — and then watch floating-point cancellation make it *worse* once ε gets too small. Every number below was produced by the NumPy reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It takes the course's own test function $f(x) = x^3$, fixes $x = 2$ (analytic derivative $3x^2 = 12$), and computes the central-difference estimate by hand at $\varepsilon = 10^{-2}$, getting 12.0001. It then tabulates the error across $\varepsilon \in \{10^{-1}, \dots, 10^{-8}\}$ to expose both the $O(\varepsilon^2)$ truncation regime and the round-off regime, locating the sweet spot. Finally it runs a multi-dimensional check on $f(\mathbf{v}) = \sum v_i^3$ and reports the max absolute and relative error that decide PASS/FAIL.

How to read this. Each idea gets a *Mini-example* box isolating it on the cleanest numbers, alongside the running x^3 example. Read this to understand the one safety net that catches a wrong `backward()`: if numeric and analytic gradients disagree, your backward is broken — no theory required.

Concepts covered, in order: the forward and central difference quotients · Taylor expansion and the $O(\varepsilon)$ vs $O(\varepsilon^2)$ truncation error · the worked x^3 estimate at $\varepsilon = 10^{-2}$ · error-vs-step table and the ε^2 law · catastrophic cancellation and the round-off floor · the sweet spot near 10^{-5} in float64 · the coordinate-wise multi-dimensional check · absolute vs relative error and the PASS criterion.

1 The setup: a derivative we already know

A *backward pass* computes a gradient analytically — by the chain rule, coded by hand or by autograd. The danger is that a hand-derived backward can be subtly wrong (a missing factor, a swapped sign) and still run without error. Gradient checking is the cheap unit test that catches this: estimate the same gradient a *second*, independent way — by poking the function — and compare.

The second way uses only *function values*, never derivatives. Pick a tiny step ε and ask how much f changes. We anchor everything on the course’s test case, where the true answer is known exactly:

$$f(x) = x^3, \quad x = 2, \quad f'(x) = 3x^2 \Rightarrow f'(2) = 3 \cdot 4 = \boxed{12}.$$

Quantity	Symbol	Value here
Test function	$f(x)$	x^3
Evaluation point	x	2
Analytic derivative	$f'(x) = 3x^2$	12
Step size	ε	swept $10^{-1} \dots 10^{-8}$
Working precision	—	float64
PASS tolerance	tol	10^{-5} (relative)

Intuition

You do not need calculus to find a slope — you need a ruler. Stand at x , take a small step right to $x + \varepsilon$ and a small step left to $x - \varepsilon$, and read off how high f climbed. Rise over run *is* the slope. Gradient checking is just this measurement, performed numerically, used as a witness against the slope your backward pass *claims*. If the witness and the claim disagree, the claim is guilty.

2 Step 1 — Two difference quotients, and why one is better

There are two natural ways to turn ”rise over run” into a formula.

Forward difference — step only to the right:

$$\frac{df}{dx} \approx \frac{f(x + \varepsilon) - f(x)}{\varepsilon}.$$

Central difference — step both ways and average:

$$\frac{df}{dx} \approx \frac{f(x + \varepsilon) - f(x - \varepsilon)}{2\varepsilon}$$

Both converge to $f'(x)$ as $\varepsilon \rightarrow 0$, but they converge at different rates. Expand f in a Taylor series around x :

$$f(x \pm \varepsilon) = f(x) \pm f'(x)\varepsilon + \frac{1}{2}f''(x)\varepsilon^2 \pm \frac{1}{6}f'''(x)\varepsilon^3 + \dots$$

Subtracting the minus-version from the plus-version, the even-order terms (including f'') *cancel*, leaving

$$f(x + \varepsilon) - f(x - \varepsilon) = 2f'(x)\varepsilon + \frac{1}{3}f'''(x)\varepsilon^3 + \dots \Rightarrow \frac{f(x + \varepsilon) - f(x - \varepsilon)}{2\varepsilon} = f'(x) + \underbrace{\frac{1}{6}f'''(x)\varepsilon^2}_{O(\varepsilon^2)} + \dots$$

So the central estimate's error is $O(\varepsilon^2)$. The forward estimate keeps the f'' term, giving error $\frac{1}{2}f''(x)\varepsilon = O(\varepsilon)$. Halving ε cuts the forward error in half but the central error by *four*.

When this is the right choice

Use the **central** difference for gradient checking. It costs one extra function evaluation per coordinate but buys an order of accuracy ($O(\varepsilon^2)$ vs $O(\varepsilon)$), which means you can verify a backward pass to many more digits with the same step. Reserve the forward difference for cases where evaluating f is so expensive that the second probe is unaffordable.

3 Step 2 — The worked estimate at $\varepsilon = 10^{-2}$

Now plug in $f(x) = x^3$, $x = 2$, $\varepsilon = 10^{-2}$. The two probe points are $x + \varepsilon = 2.01$ and $x - \varepsilon = 1.99$:

$$(2.01)^3 = 8.120601, \quad (1.99)^3 = 7.880599.$$

Their difference is the total rise across a run of $2\varepsilon = 0.02$:

$$f(x + \varepsilon) - f(x - \varepsilon) = 8.120601 - 7.880599 = 0.240002,$$

$$\left. \frac{df}{dx} \right|_{\text{central}} = \frac{0.240002}{0.02} = \boxed{12.0001}.$$

The analytic value is 12, so the central estimate is high by exactly 1.0×10^{-4} . Compare the cheaper forward difference at the same step, which uses $f(x) = 2^3 = 8$:

$$\frac{(2.01)^3 - 2^3}{0.01} = \frac{8.120601 - 8}{0.01} = \frac{0.120601}{0.01} = 12.0601,$$

an error of 6.01×10^{-2} — about $600\times$ worse. The central scheme already pays off at the first step.

Mini-example — this operation in isolation

The two error sizes are not random; the Taylor terms predict them. For $f = x^3$ at $x = 2$ we have $f''' = 6$ and $f'' = 12$, so the leading errors are

$$\text{central: } \frac{1}{6}f'''\varepsilon^2 = \varepsilon^2, \quad \text{forward: } \frac{1}{2}f''\varepsilon = 6\varepsilon.$$

At $\varepsilon = 10^{-2}$ these predict $\varepsilon^2 = 10^{-4}$ and $6\varepsilon = 6 \times 10^{-2}$ — exactly the 1.0×10^{-4} and 6.01×10^{-2} we measured. The theory and the arithmetic agree to the digit.

4 Step 3 — Error versus step: the ε^2 law, then the floor

Repeat the central estimate across a ladder of step sizes. In exact arithmetic the error would keep falling like ε^2 forever. In float64 it does not: below a certain step, subtracting two nearly-equal numbers destroys precision and the error *climbs* again.

ε	central estimate	absolute error	regime
10^{-1}	12.010000	1.000×10^{-2}	truncation ($\propto \varepsilon^2$)
10^{-2}	12.000100	1.000×10^{-4}	truncation
10^{-3}	12.000001	1.000×10^{-6}	truncation
10^{-4}	12.000000	1.001×10^{-8}	truncation
10^{-5}	12.000000	2.118×10^{-10}	sweet spot
10^{-6}	12.000000	7.892×10^{-10}	round-off rising
10^{-7}	12.000000	6.316×10^{-9}	round-off
10^{-8}	12.000000	1.173×10^{-7}	round-off (worse!)

Read the top half top-to-bottom: each $10\times$ smaller step cuts the error by $100\times$ — the signature of $O(\varepsilon^2)$. The errors $10^{-2}, 10^{-4}, 10^{-6}$ at steps $10^{-1}, 10^{-2}, 10^{-3}$ are exactly ε^2 , confirming the leading constant $\frac{1}{6}f''' = 1$ derived above. But after $\varepsilon = 10^{-5}$ the trend reverses: at $\varepsilon = 10^{-8}$ the error is 1.173×10^{-7} , *worse* than at $\varepsilon = 10^{-4}$. The minimum sits near $\varepsilon = 10^{-5}$.

Intuition

Two opposing forces fight as ε shrinks. *Truncation error* (from dropping Taylor terms) falls like ε^2 — it wants ε small. *Round-off error* grows like $\sim u/\varepsilon$, where $u \approx 10^{-16}$ is the float64 machine epsilon, because you divide a noisy near-zero difference by a tiny 2ε — it wants ε large. Their sum is a valley. The bottom of the valley, where $\varepsilon^2 \approx u/\varepsilon$, lands near $\varepsilon \approx u^{1/3} \approx 10^{-5.3}$ — precisely the 10^{-5} sweet spot in the table.

Watch out

A *smaller* ε is not automatically more accurate. Push it to 10^{-8} or below and **catastrophic cancellation** dominates: $f(x+\varepsilon)$ and $f(x-\varepsilon)$ agree in their leading digits, so their difference keeps only the few noisy trailing bits, and dividing by the tiny step amplifies that noise. Always gradient-check in `float64` (not `float32`, whose $u \approx 10^{-7}$ moves the floor up to useless levels) and keep ε near 10^{-5} .

5 Step 4 — The multi-dimensional check, coordinate by coordinate

Real gradients are vectors. The recipe is unchanged — you simply perturb *one coordinate at a time*, holding the others fixed, and apply the central difference to that slot. We use the course's vector test function

$$f(\mathbf{v}) = \sum_i v_i^3, \quad \mathbf{v} = (1, 2, 3), \quad \nabla f = (3v_1^2, 3v_2^2, 3v_3^2) = (3, 12, 27).$$

For coordinate i the numeric estimate is

$$g_i = \frac{f(\mathbf{v} + \varepsilon \mathbf{e}_i) - f(\mathbf{v} - \varepsilon \mathbf{e}_i)}{2\varepsilon},$$

where $\mathbf{e}_i$ is the i -th unit vector. Because the cubes of the untouched coordinates appear in both $f(\mathbf{v} + \varepsilon \mathbf{e}_i)$ and $f(\mathbf{v} - \varepsilon \mathbf{e}_i)$, they cancel, leaving exactly the scalar central difference of v_i^3 . At $\varepsilon = 10^{-2}$:

i	v_i	analytic $3v_i^2$	numeric g_i	absolute error
0	1	3	3.0001	1.000×10^{-4}
1	2	12	12.0001	1.000×10^{-4}
2	3	27	27.0001	1.000×10^{-4}

Every coordinate carries the same absolute error $\approx \varepsilon^2 = 10^{-4}$, because each is just the scalar v_i^3 check with $f''' = 6$. We then collapse the vector disagreement into two scalars — the worst case decides PASS/FAIL:

$$\max \text{ abs error} = \max_i |g_i - \partial_i f|, \quad \max \text{ rel error} = \max_i \frac{|g_i - \partial_i f|}{|\partial_i f| + |g_i| + 10^{-12}}.$$

At $\varepsilon = 10^{-5}$ (the sweet spot) the same machinery gives a far tighter result:

$$\max \text{ abs error} = 2.562 \times 10^{-10}, \quad \max \text{ rel error} = 4.028 \times 10^{-11}.$$

When this is the right choice

Compare the **relative** error, not the absolute one, when components differ in scale. Here $\partial_2 f = 27$ and $\partial_0 f = 3$ share the same absolute error 10^{-4} , but the relative errors are 3.7×10^{-6} and 3.3×10^{-5} — the small component is the harder test. A fixed absolute tolerance would wrongly flatter large-magnitude gradients and wrongly fail small ones.

6 Step 5 — The PASS criterion

The check returns a single boolean. Fix a tolerance — $\text{tol} = 10^{-5}$ is a standard choice with central differences in float64 — and declare:

$$\text{PASS} \iff \max_i \frac{|g_i - \partial_i f|}{|\partial_i f| + |g_i| + 10^{-12}} < \text{tol}$$

At $\varepsilon = 10^{-5}$ our vector check yields a max relative error of 4.028×10^{-11} , which is $\ll 10^{-5}$, so the test **PASSES** — the analytic gradient (3, 12, 27) is confirmed. Had the backward pass returned, say, (3, 12, **26**) for the last slot, the relative error there would be $|27 - 26|/(27 + 26) \approx 1.9 \times 10^{-2} \gg 10^{-5}$ and the check would **FAIL**, pinpointing the broken coordinate.

Watch out

A *passing* check confirms only the points you tested; it cannot prove correctness everywhere. And a *failing* check is occasionally the numeric side's fault — a non-smooth f (e.g. a kink from ReLU at 0) or a too-small ε . When a check fails, first re-run at $\varepsilon = 10^{-5}$ in float64 and away from kinks before concluding your backward is wrong.

7 The whole procedure, at a glance

#	Step	For our example	Result
1	Pick test point	$f = x^3, x = 2$	analytic $f' = 12$
2	Central difference	$\varepsilon = 10^{-2}$	12.0001
3	Truncation error	$\frac{1}{6}f''' \varepsilon^2$	1.0×10^{-4}
4	Shrink ε	$10^{-1} \rightarrow 10^{-5}$	error $\downarrow$ as ε^2
5	Floor hits	$\varepsilon \leq 10^{-6}$	round-off $\uparrow$
6	Sweet spot	$\varepsilon \approx 10^{-5}$	error 2.1×10^{-10}
7	Vectorise (one coord/step)	$f = \sum v_i^3$	grad (3, 12, 27)
8	Reduce to scalars	max abs / max rel	2.6×10^{-10} / 4.0×10^{-11}
9	Compare to <code>tol</code>	$4.0 \times 10^{-11} < 10^{-5}$	PASS

These nine rows *are* a gradient check. Run them whenever you hand-write a `backward()` or a custom `autograd.Function`; a green PASS here is the difference between trusting your gradients and debugging a silently-wrong model for a week.

8 Equation cheat-sheet

Forward difference	$\frac{f(x + \varepsilon) - f(x)}{\varepsilon}, \text{ error } O(\varepsilon)$
Central difference	$\frac{f(x + \varepsilon) - f(x - \varepsilon)}{2\varepsilon}, \text{ error } O(\varepsilon^2)$
Truncation error (central)	$\frac{1}{6}f'''(x)\varepsilon^2$
Round-off error	$\sim u/\varepsilon, \quad u \approx 10^{-16} \text{ (float64)}$
Optimal step	$\varepsilon^* \approx u^{1/3} \approx 10^{-5}$
Coordinate i estimate	$g_i = \frac{f(\mathbf{v} + \varepsilon \mathbf{e}_i) - f(\mathbf{v} - \varepsilon \mathbf{e}_i)}{2\varepsilon}$
Relative error	$\frac{ g_i - \partial_i f }{ \partial_i f + g_i + 10^{-12}}$
PASS test	$\max_i(\text{rel error}) < \text{tol}$

Appendix A Reference implementation

Every number above is reproducible from the following program (NumPy in float64; the PyTorch version from the course is structurally identical). Change f , x , or ε to explore the truncation/round-off trade-off.

```
import numpy as np

# ---- scalar check: f(x)=x^3 at x=2, analytic f'(x)=3x^2 ----
f = lambda x: x**3
x = 2.0
analytic = 3 * x**2                                # 12.0

eps = 1e-2
central = (f(x+eps) - f(x-eps)) / (2*eps)          # 12.000099999... -> 12.0001
forward = (f(x+eps) - f(x)) / eps                  # 12.0601
print(central, abs(central - analytic))             # 12.0001 , 1.0e-4
print(forward, abs(forward - analytic))             # 12.0601 , 6.01e-2

# ---- error vs step: O(eps^2) then round-off floor ----
for k in range(1, 9):
    e = 10.0**(-k)
    c = (f(x+e) - f(x-e)) / (2*e)
    print(f"eps=1e-{k} err={abs(c-analytic):.3e}")
# 1e-2 -> 1e-4, 1e-3 -> 1e-6 (eps^2 law); minimum near 1e-5;
# 1e-8 -> 1.17e-7 (cancellation makes it WORSE)

# ---- multi-dim gradient check: f(v)=sum(v^3) ----
def numgrad(f, v, eps=1e-5):
    g = np.zeros_like(v)
    for i in range(len(v)):
        vp = v.copy(); vp[i] += eps
        vm = v.copy(); vm[i] -= eps
        g[i] = (f(vp) - f(vm)) / (2*eps)           # one coordinate at a time
    return g

F = lambda v: np.sum(v**3)
v = np.array([1.0, 2.0, 3.0])
analytic_grad = 3 * v**2                          # [ 3. 12. 27.]
numeric_grad = numgrad(F, v, eps=1e-5)

abs_err = np.abs(numeric_grad - analytic_grad)
rel_err = abs_err / (np.abs(analytic_grad) + np.abs(numeric_grad) + 1e-12)
print("analytic:", analytic_grad)                  # [ 3. 12. 27.]
print("numeric :", numeric_grad)                    # [ 3. 12. 27.]
print("max abs err:", abs_err.max())                # 2.562e-10
print("max rel err:", rel_err.max())                # 4.028e-11

tol = 1e-5
print("PASS" if rel_err.max() < tol else "FAIL")    # PASS
```